UNIKERNELS / SECURITY CONFERENCE

UNIKERNELS: SECURE APPLICATION DEPLOYMENT

Unikernels: single-purpose operating systems built around your application.

UNIKERNELS | NANOVMS / NANOS | LIVE DEMO

Attackers are exploiting the gaps.

Recent reporting shows that vulnerability exploitation is a primary path into organizations.

31%

OF BREACHES

Vulnerability exploitation was the initial access vector in Verizon's 2026 DBIR.

Verizon DBIR 2026

884

NEW EXPLOITED CVES

VulnCheck recorded first-time exploitation evidence during 2025.

VulnCheck 2026

29%

AT OR BEFORE DISCLOSURE

Of those 884, exploitation was observed on or before CVE publication.

VulnCheck 2026

Sources: Verizon 2026 DBIR; VulnCheck State of Exploitation 2026 (2025 observations).

RCE is an entry point, not the entire attack.

A flaw may give an attacker code execution. The surrounding operating system determines what comes next.

01 INITIAL COMPROMISE

Application flaw

Command injection, unsafe deserialization, memory corruption, or another exploitable bug.

02 POST-EXPLOITATION

Available OS capabilities

A shell, child processes, utilities, credentials, and network access may enable lateral movement.

03 ARCHITECTURAL CHOICE

Remove what is unnecessary

Unikernels reduce OS features and execution paths, but the application still needs secure code.

48,174 CVEs were published in 2025. Not every CVE is RCE, and published exploit code ≠ confirmed exploitation.

Source: VulnCheck 2026 Exploit Intelligence Report • 2025 calendar year

Reduce the attack surface by design.

Container escapes, RCE vulnerabilities, and data exposures challenge traditional deployment assumptions.

ELIMINATE

Unneeded OS services, shells, administrative tooling, and secondary processes.

SPECIALIZE

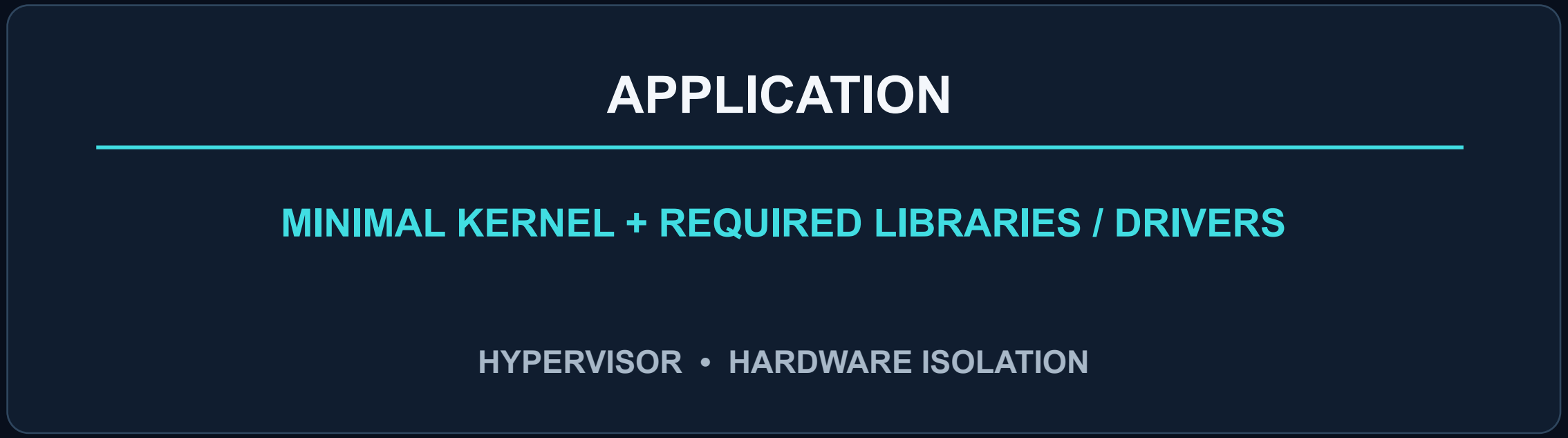
A single-purpose OS image containing your app and the runtime components it requires.

CONTAIN

Smaller attack surface and fewer post-exploitation tools, backed by VM isolation and least privilege.

What is a unikernel?

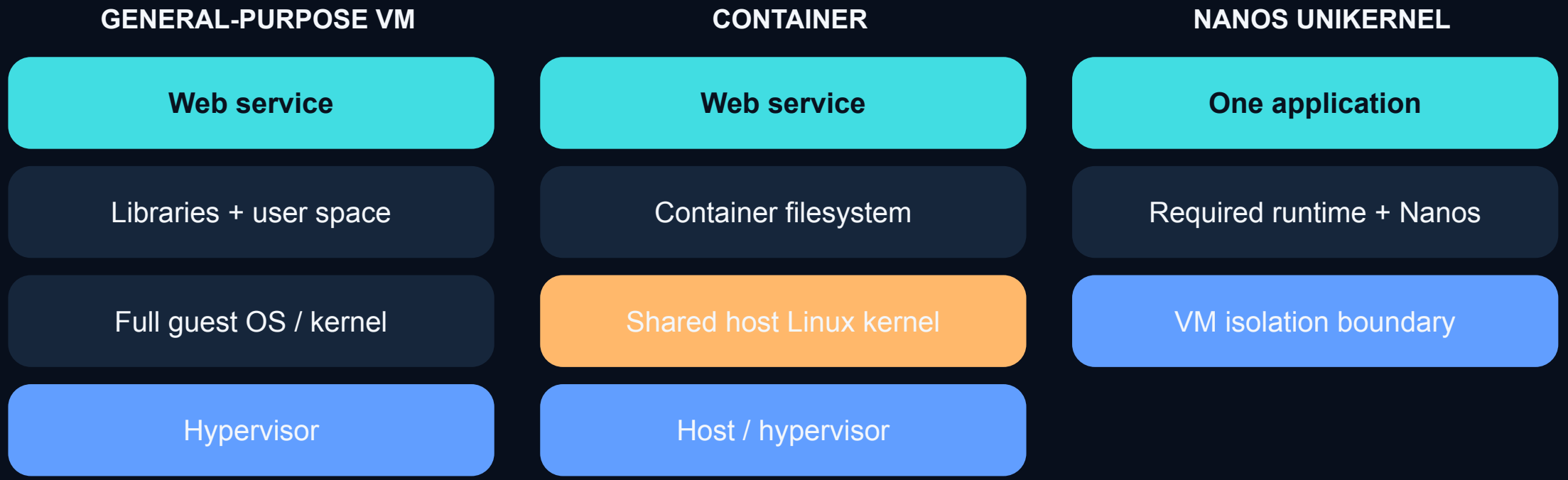
A single-purpose operating system image tailored to one application and its dependencies.



ARCHITECTURE

Three ways to run one web service

A useful mental model, not a claim that every container image includes all these utilities.



Attack surface is not the same as exploitability

Reduced exposure vectors and blast radius are goals, not automatic guarantees.

01	ENTRY	HTTP endpoints, parsers, dependencies, protocol handlers.
02	EXPLOIT	Memory corruption, deserialization, injection, authorization bugs.
03	POST-EXPLOIT	Shell, tools, process launch, persistence, lateral movement.
04	BOUNDARY	VM/hypervisor escape, network policy, cloud identities, secrets.

NanoVMs: a commercial path to unikernels

Nanos is the single-application kernel. Ops builds, packages, and runs Nanos images.

NANOS

Single-process design with threads. No general-purpose users, shell, SSH, or launching other programs.

OPS

CLI for building and running workload-specific unikernel images, locally and in supported clouds.

COMMERCIAL REALITY Open-source Nanos exists. NanoVMs-provided binaries and support can involve commercial terms.

EXPECTED EXPLOIT OUTCOMES

What “RCE” actually means here

Three attack paths with three different conclusions.

Remote shell / command injection	Shell-based payloads often fail when no shell or child-process model exists.	CONSTRAINED
Arbitrary code inside the app	Memory corruption or interpreter/runtime exploit may still execute attacker logic.	POSSIBLE
Application-level compromise	Broken auth, SSRF, SQL injection, data exfiltration still matter.	POSSIBLE

Same vulnerable service. Different runtime.

Demonstrate a missing execution primitive, then demonstrate a surviving application flaw.

LINUX / STANDARD CONTAINER

GET /run?cmd=id
→ invokes /bin/sh -c id
→ command output / identity

NANOS UNIKERNEL

GET /run?cmd=id
→ process launch unavailable
→ request returns error

CONTROL: /preview?name=owned still reflects unsafe HTML in both.

The deliberately vulnerable Go handler

Harmless command injection illustration. Never expose this demo app on a public network.

```
func runCommand(w http.ResponseWriter, r *http.Request) {
        command := r.URL.Query().Get("cmd")
//        …..redacted for brevity…..
        cmd := exec.CommandContext(ctx, "/bin/sh", "-c", command)
        output, err := cmd.CombinedOutput()
        //      …..redacted for brevity…..
        if err != nil {
                w.WriteHeader(http.StatusInternalServerError)
                _, _ = fmt.Fprintf(w, "exec %q: %v\n", command, err)
        }
        _, _ = w.Write(output)
}
```

COMMAND SPAWNING ≠ ALL FORMS OF CODE EXECUTION

A reproducible local runbook

Preparation on Linux with KVM and a working Ops installation.

```
# Build the demo binary for the target architecture
make build

# Standard Linux process
make run-native PORT=9090
curl "http://127.0.0.1:9090/exec?cmd=id"

# Nanos: run in another terminal (port forwarding via Ops)
make run-unikernel PORT=9090
curl "http://127.0.0.1:9090/exec?cmd=id"
```

IMPORTANT Run one environment at a time. Verify port mapping and runtime compatibility beforehand.

What the attacker loses. What they keep.

A smaller OS removes common attacker primitives, but application exposure remains.

LOSES BY DESIGN

Typical shell workflows, OS command chains, SSH pivoting, installing tools, launching secondary services.

MAY STILL CONTROL

Application data and endpoints, in-process runtime behavior, outbound requests, exposed credentials and files.

Boundary matters: Nanos still has a kernel, networking code, and a hypervisor.

Security is architecture, not a checkbox

Operational and implementation details decide whether the theoretical benefits survive production.

Kernel hardening	Verify ASLR, NX/W^X, canaries, updates for your version
Visibility	Design logs, metrics, crash analysis, and debugging without SSH
Compatibility	Test syscalls, language runtime, file access, DNS, TLS
Deployment	VM lifecycle, patching, rollbacks, secrets, and cloud integration
Containment	Limit egress, identity permissions, reachable peers, data access

WHERE IT FITS

Great fit for single-purpose stateless services

Avoid force-fitting a unikernel into workloads that depend on a full Unix environment.

STRONG CANDIDATES

Small HTTP APIs, network appliances, worker services, Go/Rust services, tightly scoped edge endpoints.

HIGHER FRICTION

Complex multi-process stacks, interactive debugging, heavy OS tooling, dynamic plugins, special device drivers.

Start with one well-contained service and measure the operational cost.

CLOSING

Deploy only what the application needs.

Reduce exposure. Constrain impact. Keep securing the code.

REDUCE → ISOLATE → VERIFY

Single-purpose unikernels reduce the attack surface and can limit blast radius, but they do not eliminate RCE or data exposure.

Sources and further reading

Primary vendor documentation plus independent unikernel security research.

Nanos kernel README	github.com/nanovms/nanos
Ops build-and-run tool	github.com/nanovms/ops
Ops documentation	docs.ops.city/ops
NanoVMs security overview	nanovms.com/security
NCC Group, Assessing Unikernel Security	nccgroup.com/media/tgsj1css/_ncc_group-assessing_unikernel_security.pdf
Talbot et al., A Security Perspective on Unikernels (2020)	doi.org/10.1109/CyberSecurity49315.2020.9138883

ABOUT THE SPEAKER

Keep building. Keep questioning.

Security by subtraction is only the beginning.

ioneyed

Sr. SINOPS Engineer / Systems & Security Architecture
*(**S**oftware, **I**nfrastructure, **N**etworking, **O**perations, **P**latform, **S**ecurity Engineer)*

GITHUB

github.com/ioneyed

TALK REPOSITORY

github.com/ioneyed/qcc-0x4-unikernel

UP NEXT / SNEAK PEEK

DREY

Exploring unikernel and fixed-process operating system design.

More to come.

SCAN FOR TALK REPO